DD2424 Assignment 1 Bonus

Simon Yu

April 2026

1 Improve performance of the network

In this assignment I will be testing different approaches to improve the performance of the network one by one to see how the different methods fair against each other. The parameters:

`lam=0.1, n_epochs=40, n_batch=100, eta=.001` will be used as the initial parameters since it resulted in the best result in the ordinary assignment.

1.1 Use all available training data

First thing that was done was using all available training data for training. data_batch_1-5 was used except for 1000 images from data_batch_5 which was reserved for the validation set. This increased the amount of training images from 10000 to 49000 and increased the accuracy on the test set from 39.22% to 40.85% using the same parameters.

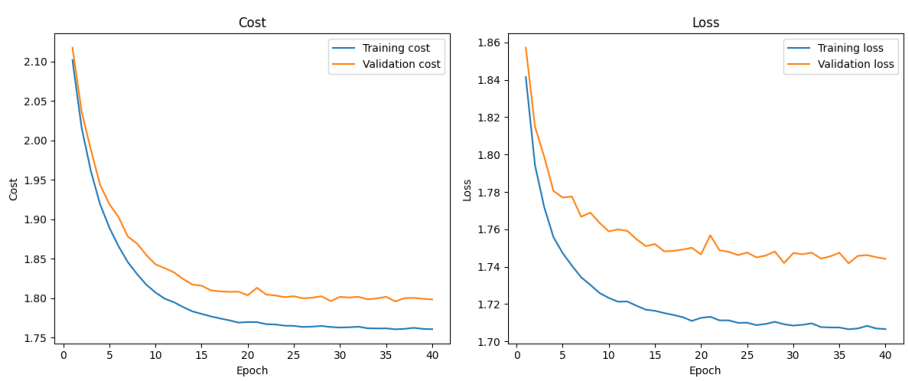

Figure 1: raining and validation cost (left) and loss (right) over epochs after using all available training data

1.2 Grid search

Secondly, a grid search was performed to test different values of `lam, eta and n_batch`. The values that was tested were:

```
lam_values = [0, 0.01, 0.1, 0.2]
eta_values = [0.0001, 0.0005, 0.001, 0.005]
batch_values = [50, 100, 200]
```

In the end `lam=.1, eta=.001, and n_batch=50` gave the best result with an test accuracy of 39.35% which didn't improve much from the initial 39.22%. This may be due to parameters from the beginning already begin quite good.

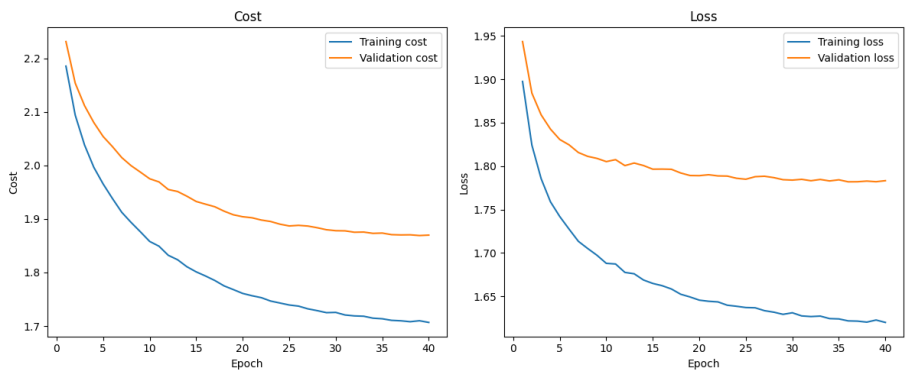

Figure 2: raining and validation cost (left) and loss (right) over epochs after grid searching for best parameters

1.3 Decaying learning rate

Thirdly, a decaying learning rate was implemented. Now the eta is decreased by a factor 10 after n-epochs. Using a low number of epochs between the decays led to worse score than in the beginning since the low learning rate that it ended with converged to slowly to improve the score in the fixed 40 epochs. But setting the parameters to decrease learning rate after 35 epochs slightly improved the performance to 39.3%.

From the small test that I have done the technique that seemed to improve the network the most is simply by using a larger training set.

2 Multiple binary cross-entropy losses

In this part I will implement multiple binary cross-entropy losses with a sigmoid activation function and compare it to the previous cross entropy loss and softmax

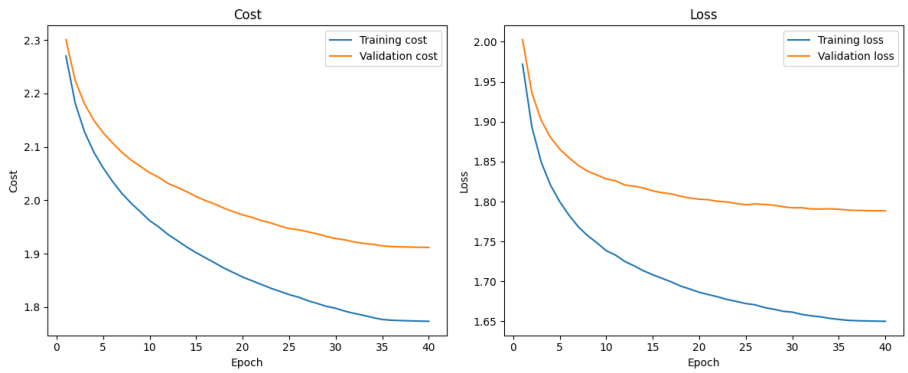

Figure 3: raining and validation cost (left) and loss (right) over epochs with a decay of learning rate after 35 epochs

activation function. The expression for $\partial l_{\text{multiple bce}}/\partial s$ can be derived by:

$$p = \sigma(s) = \frac{1}{1+e^{-s}}$$

$$l_{\text{multiple bce}} = -\frac{1}{K}\sum_{k=1}^{K}\left[(1-y_k)\log(1-p_k) + y_k\log(p_k)\right]$$

$$\frac{\partial p}{\partial s} = p(1-p)$$

$$\frac{\partial l_{\text{multiple bce}}}{\partial p} = -\frac{1}{K}\left(\frac{y}{p} - \frac{1-y}{1-p}\right)$$

$$= -\frac{1}{K}\left(\frac{y(1-p) - p(1-y)}{p(1-p)}\right)$$

$$= -\frac{1}{K}\left(\frac{y-p}{p(1-p)}\right)$$

Using the chain rule,

$$\frac{\partial l_{\text{multiple bce}}}{\partial s} = \frac{\partial l_{\text{multiple bce}}}{\partial p}\frac{\partial p}{\partial s}$$

$$= -\frac{1}{K}\left(\frac{y-p}{p(1-p)}\right)p(1-p)$$

$$= -\frac{1}{K}(y-p)$$

$$= \frac{1}{K}(p-y)$$

Using the parameters: `n_batch=100, eta=.01, lam=.1, and n_epochs=40` for both methods resulted in a accuracy of 37.93% and 37.67% for the softmax and sigmoid activation function respectively. Therefore we can see that the sigmoid function did not manage to perform better than the softmax function(atleast not with any of the parameters that I tried with). But in Figure 4 it can be seen that the sigmoid function with multiple BCE loss is far less prone to overfitting.

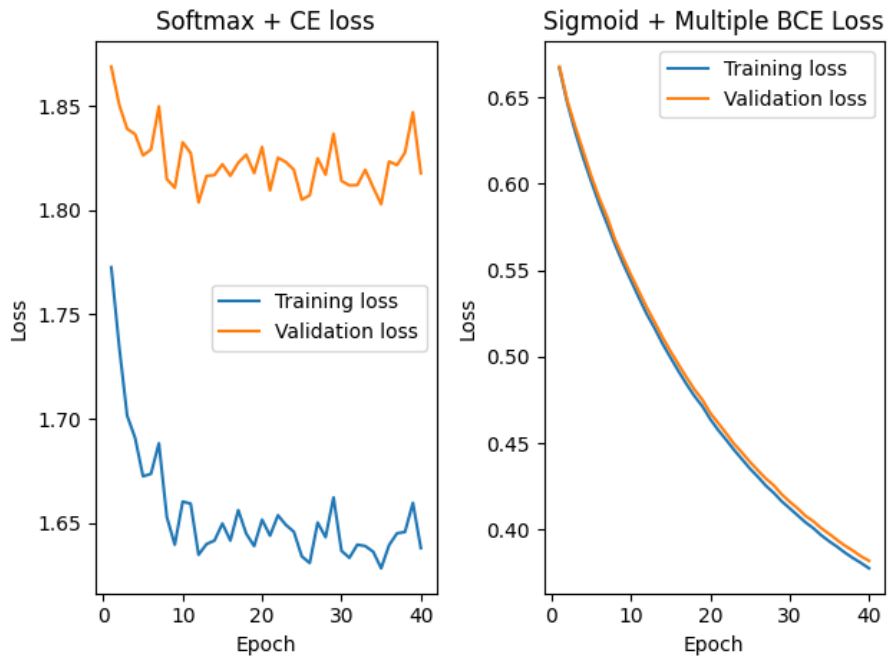

Figure 4: loss function for Softmax + CE and Sigmoid + Multiple BCE

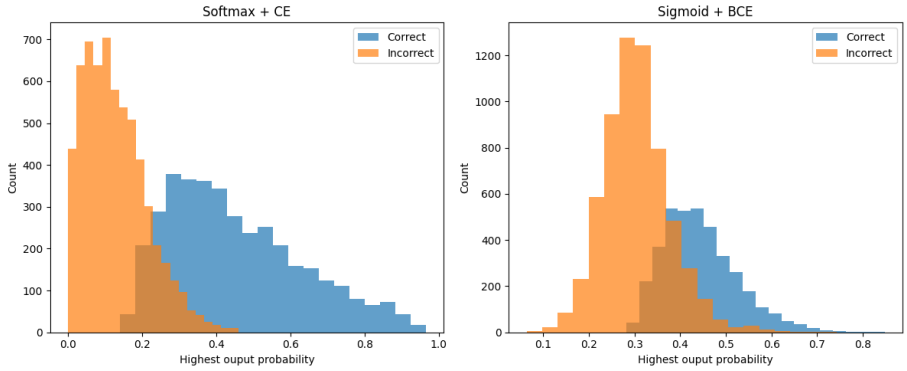

Figure 5: Histogram comparison between softmax + CE and sigmoid + BCE